

Topic 6.8

6.8: Virtual Destructors and Cloning

Whenever a class has at least one virtual function, the destructor should be virtual. Failure to designate destructors as virtual in such cases may result in the base class destructor being invoked *only*, instead of the derived class's destructor followed (automatically) by the base class's destructor. This may fail to deallocate all space associated with the object, or other errors.

Constructors *cannot* be virtual. (Think about it: constructors are called to create new objects. However, if you don't know what type of object you're trying to create, how do you know which constructor to invoke?)

Nevertheless, there may be times when a virtual constructor would be nice. Sometimes we have a pointer to an object (in a hierarchy), and we don't know which specific type of object the pointer points to, but we want to duplicate the object we have the pointer to (see a real example in the Linked List example). The standard technique in these situations is to include a *clone* function, virtual of course, which returns a pointer to a duplicate object. The duplicate is usually obtained by a copy constructor that takes a reference to a class object.

The point of the *clone* virtual function is illustrated by the following:

```
BaseClass*p;
BaseClass*pdup;

// p points at a BaseClass object, or an object of
// any class derived from BaseClass. We don't
// know exactly which type of object p points to,
// but we want to duplicate that object, and
// make pdup point at the duplicate.

pdup = p->clone();
```

The following example has clone functions. I've also added copy constructors to each class in the hierarchy. This makes our the function trivial: simply dynamically allocate an object of the appropriate type and initialize that object using the copy constructor, with the original object as the argument to the copy constructor.

```
// MODULE:      inherit6.cpp
// PROGRAMMER:  Gary J. Blair
// DATE:        96/04/11
// COMPILER:    MS Visual C++ 2008 Express Edition
// REQUIRED:     inherit6.cpp
//
// PURPOSE:
```

```

// Illustrate inheritance.
//

#include <iostream>
#include <stdlib.h>
#include <string.h>

using namespace std;

class Thing      // Thing is the base of all
                // our objects.
{
    public:
    Thing() { strcpy(name,""); };
    Thing( Thing& t ) { strcpy(name,t.name); };
    Thing( char * thing_name )
    { strcpy(name,thing_name); };
    virtual Thing *clone()
    { return new Thing(*this); };
    virtual void print();

    protected:
    char name[80];
};

void Thing::print()
{
    cout << "name:" << name;
}

class Person : public Thing
{
    public:
    Person();
    Person( char *person_name, int person_age );
    Person( Person& p );
    virtual Thing *clone()
    { return new Person(*this); };
    void print();

    protected:
    int age;
};

Person::Person() : Thing()
{
    age = -1;
}

Person::Person
( char *person_name, int person_age ) :
Thing(person_name)
{
    age = person_age;
}

Person::Person( Person& p ) : Thing(p.name)
{
    age = p.age;
};

void Person::print()
{
    Thing::print();
    cout << " age: " << age;
}

```

```

class Politician : public Person
{
    public:
    Politician( char *person_name, int person_age,
    char *pol_party );
    Politician( Politician& p );
    virtual Thing *clone()
    {
        return new Politician(*this);
    };
    void print();

    protected:
    char party[40];
};

Politician::Politician
( char *person_name, int person_age,
char *pol_party )
: Person(person_name,person_age)
{
    strcpy(party,pol_party);
}

Politician::Politician
( Politician& p ) :
    Person(p.name,p.age)
{
    strcpy(party,p.party);
}

void Politician::print()
{
    Person::print();
    cout << " party:" << party;
}

int main(void)
{
    Thing      t("Cessna 182");
    Person      me("Gary J. Blair",37);
    Politician  president("Bill Clinton",
46,"Democrat");
    Thing      *things[3] = {&t, &me, &president};
    Thing      *cloned_things[3];
    int        i;

    for (i=0; i<3; ++i)
    {
        cloned_things[i] = things[i]->clone();
        cout << "Cloning\n ";
        things[i]->print();
        cout << "\nyields\n ";
        cloned_things[i]->print();
        cout << "\n\n";
    }

    for (i=0; i<3; ++i)
        delete cloned_things[i];

    return 0;
}

```

Output from inherit6.cpp:

```
Cloning
name:Cessna 182
yields
name:Cessna 182
Cloning
name:Gary J. Blair age: 37
yields
name:Gary J. Blair age: 37
    Cloning
name:Bill Clinton age: 46 party:Democrat
yields
name:Bill Clinton age: 46 party:Democrat
```

Module 6 (<https://onlinelearning.berkeley.edu/courses/2291660/pages/module-6>)